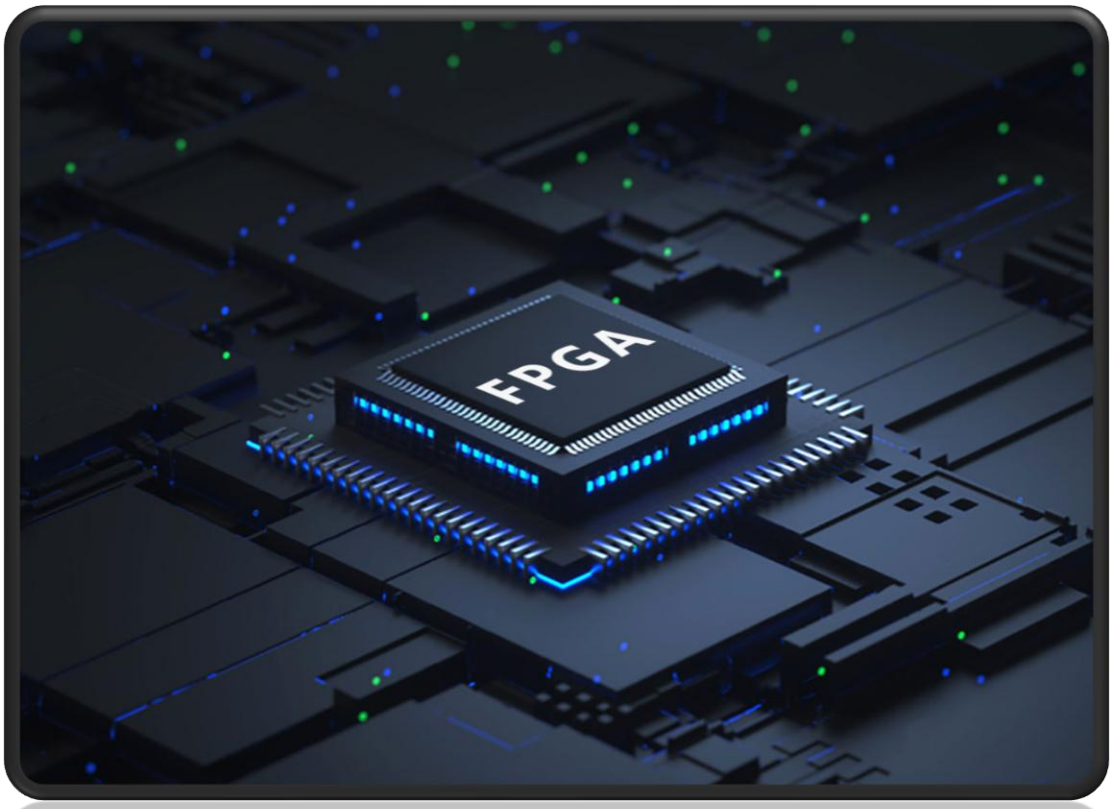
POLYTECH SORBONNE

西安電子科技大學 XIDIAN UNIVERSITY

RAPPORT DE PROJECT

PROCESSEUR MONO-CYCLE ET MULTI-CYCLE

KE YU et HONGYI CAI



Préface

Cette expérience consiste à implémenter un processeur ARM7TDMI simplifié selon les architectures mono-cycle et multi-cycles en VHDL, afin d'en approfondir le fonctionnement. Le rapport présentera une analyse méthodique des aspects techniques clés, tout en validant les performances comparatives des deux architectures par simulation fonctionnelle et tests matériels sur FPGA.

Table des matières

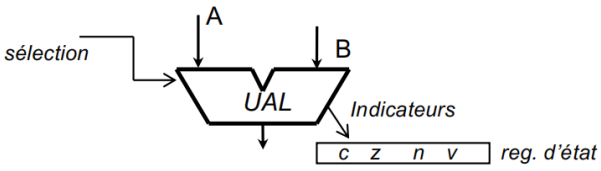
Préface 2
PROCESSEUR MONO-CYCLE 4
PARTIE 1 – UNITE DE TRAITEMENT 4
Unité Arithmétique et Logique (UAL ou ALU) 4
Ban de registre 5
Unité de traitement 6
Multiplexeur 2 vers 1 7
Extension de signe 8
Mémoire de données 8
Assemblage Unité de Traitement 9
PARTIE 2 – UNITE DE GESTION DES INSTRUCTIONS 10
Mémoire d'instructions 10
PC Extender 11
PARTIE 3 – UNITE DE CONTRÔLE 12
Décodeur d'instructions 12
Valeurs des commandes 13
PARTIE 4 – ASSEMBLAGE ET VALIDATION DU PROCESSEUR 14
PARTIE 5 : TEST DU PROCESSEUR SUR CARTE FPGA 15
PROCESSEUR MULTI-CYCLE 16
PARTIE 1 : FONCTIONNEMENT DU CHEMIN DE DONNEES + MEMOIRE 17
Controleur d'interruption vectorisé (VIC) 18
Current Processor Status Register (CPSR) 18
PARTIE 2 : LA MACHINE A ETATS 19
PARTIE 3 : CODAGE VHDL DE LA MACHINE A ETATS 21
PARTIE 4 : RAJOUTER UN PERIPHERIQUE UART AU PROCESSEUR 25

PROCESSEUR MONO-CYCLE

Cette section détaille l'architecture du processeur mono-cycle. Composé d'éléments fondamentaux tels que registres, multiplexeurs, mémoire et unité arithmético-logique, ces composants sont intégrés de manière cohérente pour former les trois blocs fonctionnels principaux : unité de traitement, unité de gestion d'instructions et unité de contrôle.

PARTIE 1 – UNITE DE TRAITEMENT

Unité Arithmétique et Logique (UAL ou ALU)



L'unité Arithmétique et logique se compose de deux entrées de données et d'une entrée de contrôle qui indique l'opération codée. Dans la sortie, nous trouvons le résultat de l'action effectuée et 4 signaux appelés « drapeaux » qui fournissent des informations sur le travail effectué, mais ici nous simplifions en ne faisant que la sortie du bit d'indicateur N et du bit d'indicateur Z, qui représentent respectivement les valeurs positives et négatives et nulles du résultat. Peaufiné et traduit en versions chinoise et française, le langage est concis et courant, et l'utilisation des numéros de série est interdite

Le fonctionnement de l'opération peut être catégorisé par quelques morceaux de code simples

```
case OP is
    when "00" => result <= std_logic_vector(signed(A) + signed(B));     -- ADD
    when "01" => result <= B;                                           -- B
    when "10" => result <= std_logic_vector(signed(A) - signed(B));     -- SUB
    when "11" => result <= A;                                           -- A
    when others => result <= (others => '0');
end case;
```

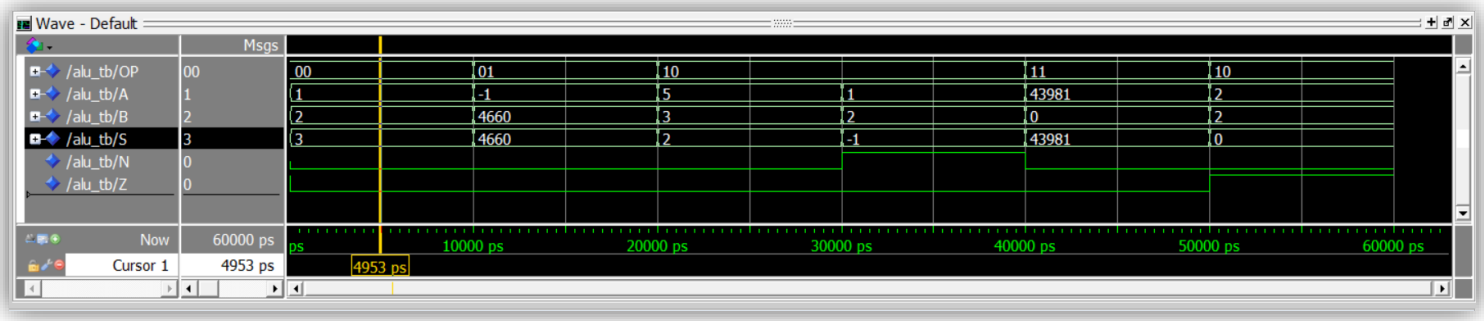
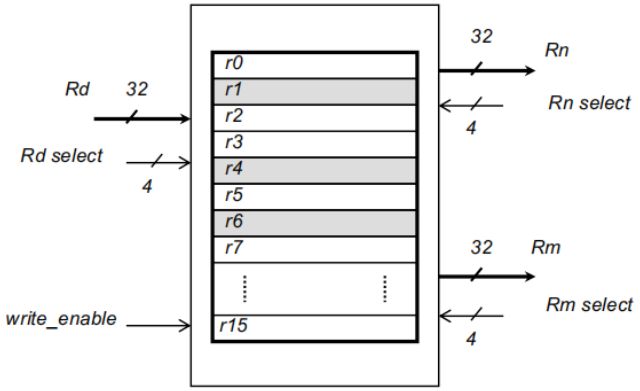


Figure 1 : Simulation de l'Unité Arithmétique et Logique

Les résultats de cette simulation peuvent être analysés conjointement avec le tableau suivant

	OP	A	B	opération	S	N	Z
00	**ADD**	1	2	S=A+B	3	0	0
01	**B**	-1	4460	S=B	4460	0	0
10	**SUB**	5	3	S=A-B	2	0	0
10	**SUB**	1	2	S=A-B	-1	1	0
11	**A**	-43981	0	S=A	43981	1	0
10	**SUB**	2	2	S=A-B	0	0	1

Ban de registre



Le banc possède 16 registres de 32 bits. Ce groupe de registres est utilisé pour regrouper les données lues à partir de la mémoire et est prêt pour le calcul. L'important pour écrire ce composant est de définir le type de données d'un groupe de registres, ce qui nécessite l'utilisation de l'instruction « type » pour définir 16 types de données de 32bits

```
type table is array(0 to 15) of std_logic_vector(31 downto 0);
```

- Pour les opérations de lecture, nous utilisons des lectures asynchrones, c'est-à-dire que nous n'avons pas besoin que le système soit toujours impliqué dans le CLK
- Pour les opérations d'écriture, nous prenons des écritures synchrones, qui sont contrôlées par le signal d'activation d'écriture WE
- Processus de lecture : Une fois que les adresses de lecture RA et RB sont modifiées, les ports de sortie BUSA et BUSB sont immédiatement mis à jour avec les valeurs des registres correspondants.
- Processus d'écriture : Étant donné les données BUSW du bus d'entrée et l'adresse d'écriture RW, une fois que le signal d'activation d'écriture WE est défini à 1, les données seront écrites dans le registre correspondant sur le front montant suivant.

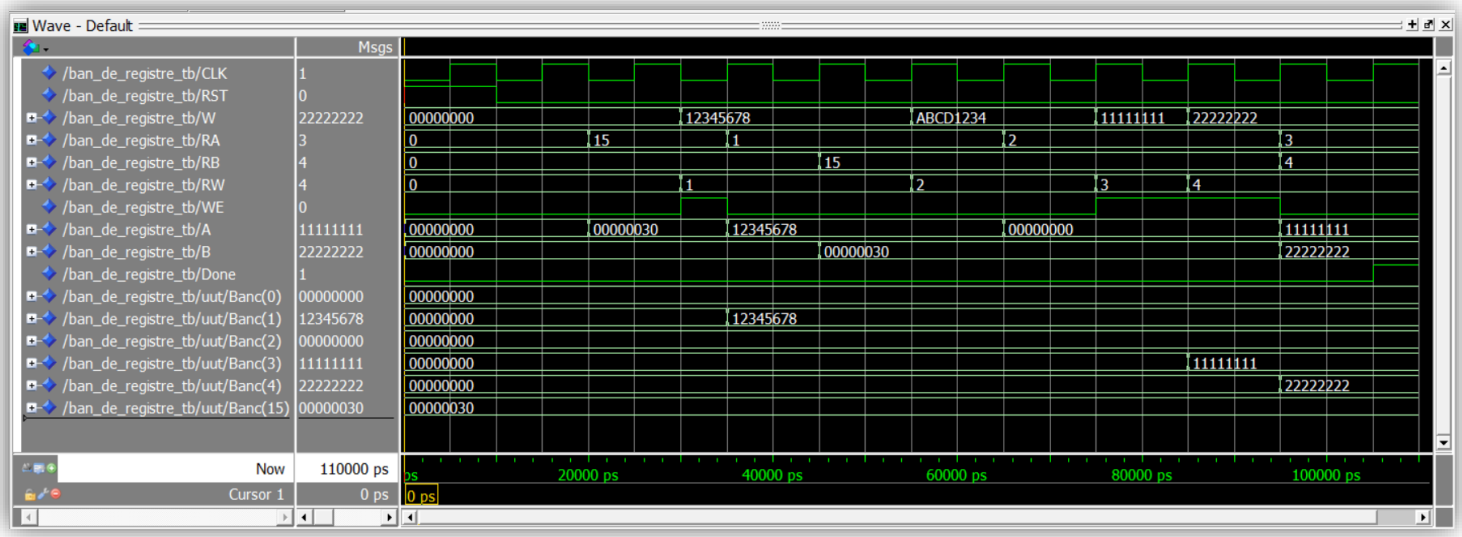
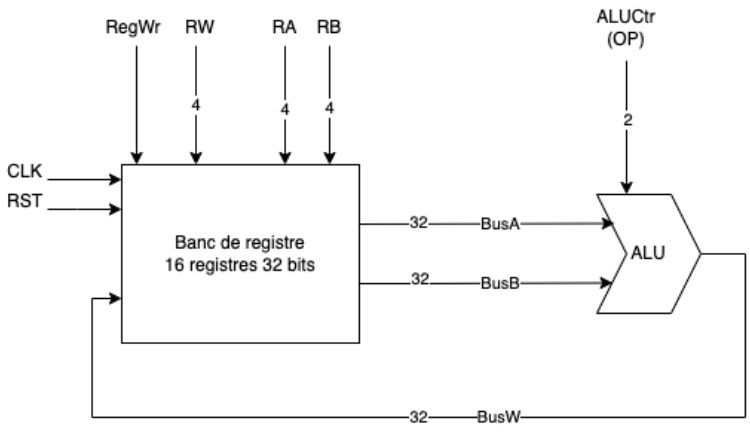


Figure 2 : Simulation du banc de registres

Unité de traitement

Assembler l'UAL et le banc comme sur le schéma ci-dessous pour valider l'unité de traitement.



Écrire un banc de test et un script de simulation pour valider par simulation le bon fonctionnement des opérations suivantes :

- ***R(1) = R(15)***
- ***R(1) = R(1) + R(15)***
- ***R(2) = R(1) + R(15)***
- ***R(3) = R(1) – R(15)***
- ***R(5) = R(7) – R(15)***

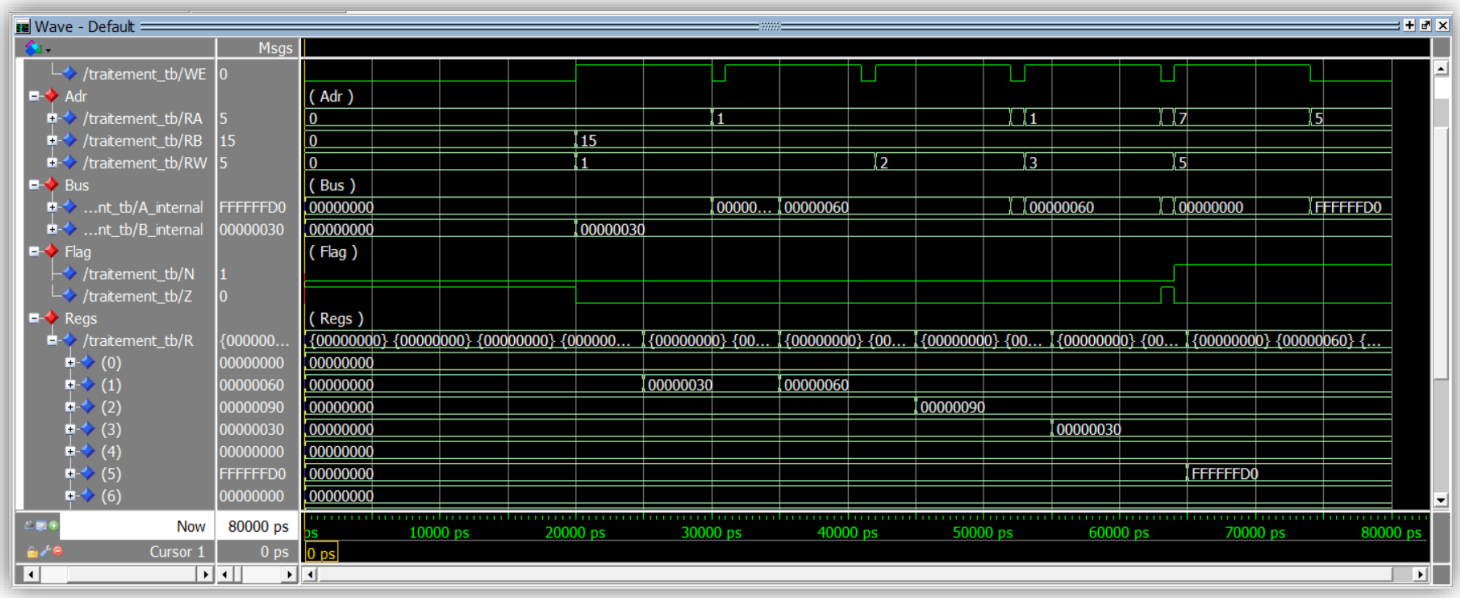


Figure 3 : Simulation de l'Unité de traitement

REMARQUE : LA VALEUR PAR DEFAUT DU REGISTRE R15 EST 0X00000030 LORS DE L'INIT - IALISATION

Multiplexeur 2 vers 1

Ce multiplexeur dispose d'un paramètre générique N fixant la taille des données en entrée et en sortie

```vhdl
entity mux21 is
    generic( N: integer := 32 );
    Port (
        A, B : in std_logic_vector(N-1 downto 0);    -- 32位输入总线
        S    : out std_logic_vector(N-1 downto 0);   -- 32位输出总线
        COM  : in std_logic                          -- data select
    );
end entity;

architecture Behavioral of mux21 is
begin
    S <= A when (COM = '0') else
         B;
end architecture;
```

Extension de signe

Ce module permet d'étendre en sortie sur 32 bits le signe d'une entrée codée sur N bits. Il possède donc un paramètre générique fixant la valeur de N.

```vhdl
entity extension is
    generic( N: integer := 8 );
    Port (
        E   : in std_logic_vector(N-1 downto 0);  -- N 位输入总线
        S   : out std_logic_vector(31 downto 0)   -- 32位输出总线
    );
end entity;

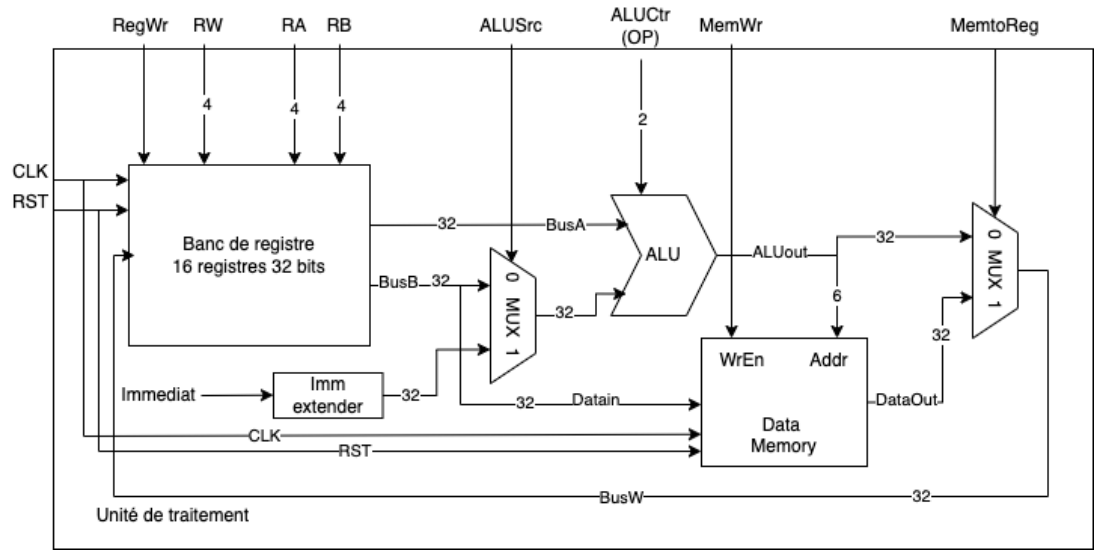
architecture Behavioral of extension is
begin
    S <= std_logic_vector(resize(signed(E), 32));
end architecture;
```

Mémoire de données

Cette mémoire permet de charger et stocker 64 mots de 32 bits. Ce composant est similaire à un registre en ce sens qu'il doit définir un type de données

```vhdl
type RAM64x32 is array(0 to 63) of std_logic_vector(31 downto 0);
```

Assemblage Unité de Traitement



L'unité de traitement d'assemblage est obtenue en connectant les composants précédemment écrits selon le schéma ci-dessus

Écrire un banc de test et un script de simulation pour valider le bon fonctionnement de :

- ***L'addition de 2 registres***
- ***L'addition d'1 registre avec une valeur immédiate***
- ***La soustraction de 2 registres***
- ***La soustraction d'une valeur immédiate à 1 registre***
- ***La copie de la valeur d'un registre dans un autre registre***
- ***L'écriture d'un registre dans un mot de la mémoire.***
- ***La lecture d'un mot de la mémoire dans un registre.***

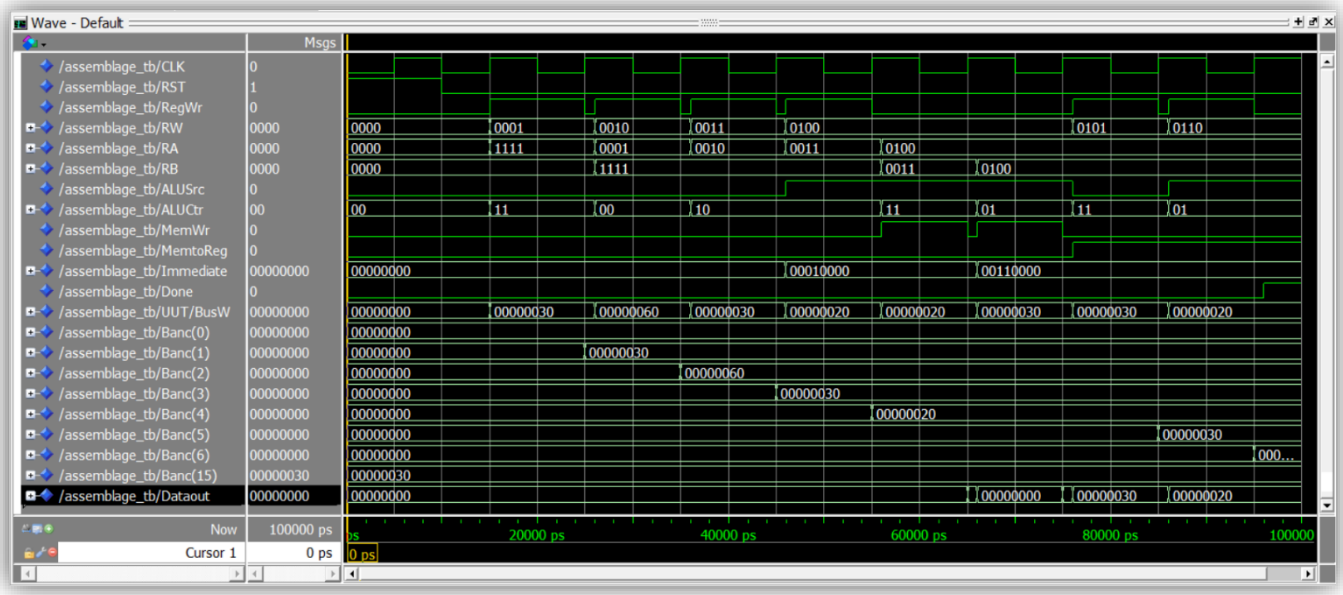
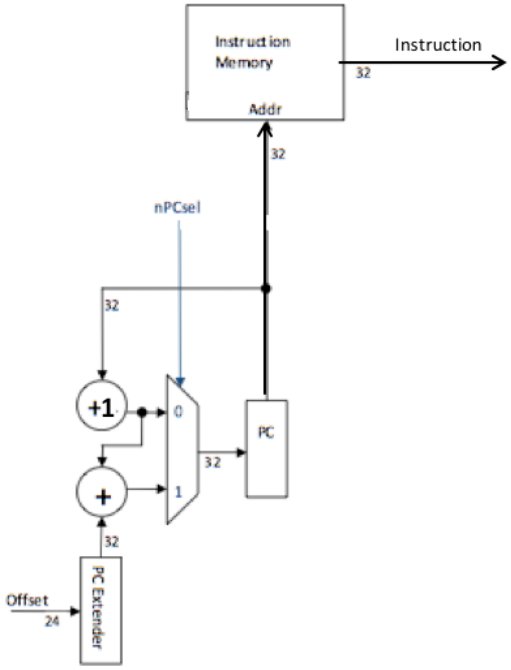


Figure 4 : Simulation de l'Assemblage de l'Unité de Traitement

PARTIE 2 – UNITE DE GESTION DES INSTRUCTIONS



Les principales fonctions de cette unité sont doubles :

- L'une d'entre elles consiste à s'assurer que la commande est exécutée de manière séquentielle : nPC=0 L'adresse de l'instruction s'auto-incrémente
- L'une d'entre elles est le saut pour exécuter l'instruction : nPC=1 instruction plus Offset

Mémoire d'instructions

Cette unité contient un registre d'instructions, qui est similaire à celui du Mémoire de données, et contient le code d'instruction de la machine ARM

```
result (0):=x"E3A01010";-- 0x0 _main -- MOV R1,#0x10 -- R1 = 0x10
result (1):=x"E3A02000";-- 0x1 -- MOV R2,#0x00 -- R2 = 0
result (2):=x"E4110000";-- 0x2 _loop -- LDR R0,0(R1) -- R0 = DATAMEM[R1]
result (3):=x"E0822000";-- 0x3 -- ADD R2,R2,R0 -- R2 = R2 + R0
result (4):=x"E2811001";-- 0x4 -- ADD R1,R1,#1 -- R1 = R1 + 1
result (5):=x"E351001A";-- 0x5 -- CMP R1,0x1A -- Flag = R1-0x1A,si R1 <= 0x1A
result (6):=x"BAFFFFFB";-- 0x6 -- BLT loop -- PC =PC+1+(-5) si N = 1
result (7):=x"E4012000";-- 0x7 -- STR R2,0(R1) -- DATAMEM[R1] = R2
result (8):=x"EAFFFFF7";-- 0x8 -- BAL main -- PC=PC+1+(-9)
```

PC Extender

La fonction est d'étendre le décalage (24 bits) intercepté de l'instruction de saut à 32 bits, ce qui est utilisé pour calculer avec le code d'instruction de 32 bits.



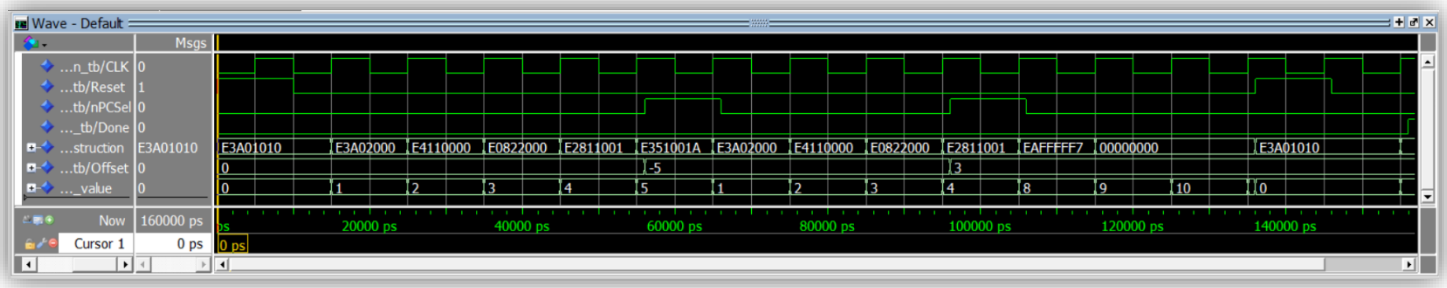
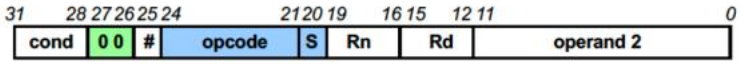


Figure 5 : Simulation de l'Unité de Gestion d'Instructions

Dans le banc de test, l'ordre de lecture des instructions, le saut d'instruction avec un offset de -5, le saut d'instruction avec un offset de 3 et la réinitialisation de l'unité ont été testés successivement.。

PARTIE 3 – UNITE DE CONTRÔLE

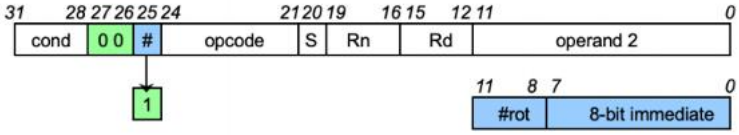
Décodeur d'instructions



Asm	Opcode
AND	0000
EOR	0001
SUB	0010
RSB	0011
ADD	0100
ADC	0101

Asm	Opcode
SBC	0110
RSC	0111
TST	1000
TEQ	1001
CMP	1010
CMN	1011

Asm	Opcode
ORR	1100
MOV	1101
BIC	1110
MVN	1111



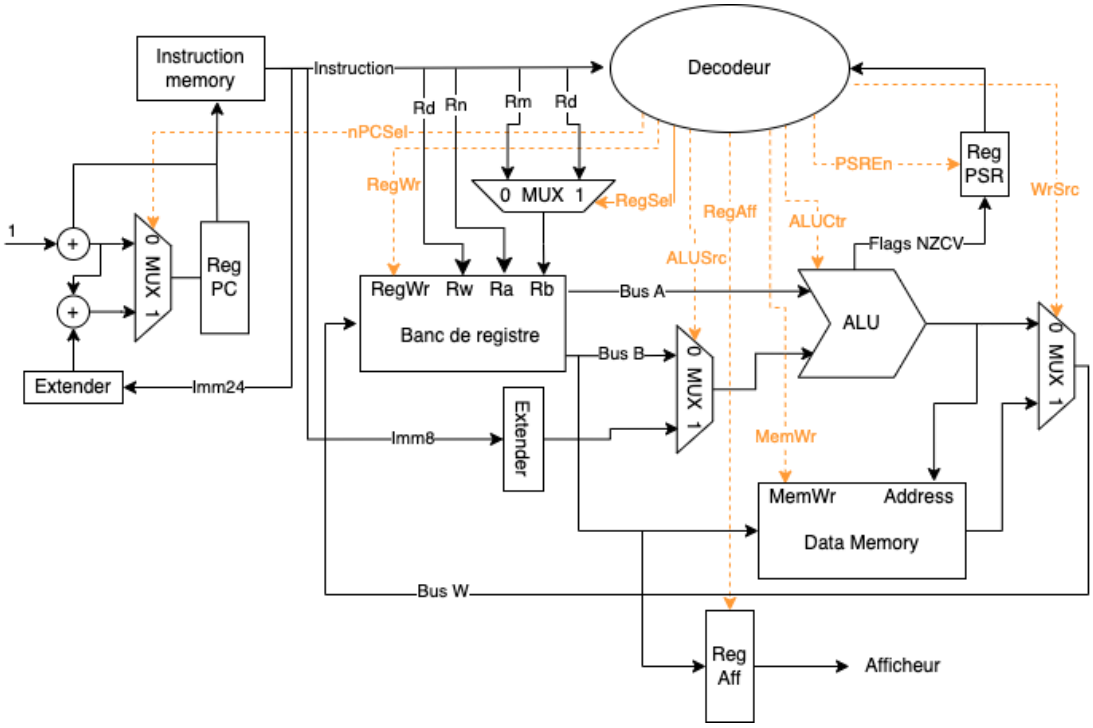
Combiné à la connaissance du code machine ARM, le code machine d'instructions est décodé :

```
-- Extract instruction fields
cond <= Instruction(31 downto 28);
I_bit <= Instruction(25);
opcode <= Instruction(24 downto 21);
S_bit <= Instruction(20);
```

```
if Instruction(27 downto 26) = "10" then
    -- Branch instruction
    if Instruction(24) = '1' then -- L bit
        if cond = "1011" then
        instr_courante <= BLT; --when cond = "1011" else BAL;
        end if;
    else
        instr_courante <= BAL;
    end if;
else
    -- Data processing instruction
    case opcode is
        when "1101" => instr_courante <= MOV;
        when "0100" =>
            if I_bit = '1' then
                instr_courante <= ADDi;
            else
                instr_courante <= ADDr;
            end if;
        when "1010" => instr_courante <= CMP;
        when "0101" => instr_courante <= LDR;
        when "0110" => instr_courante <= STR;
        when others => instr_courante <= MOV; -- Default
    end case;
end if;
```

Valeurs des commandes

Une fois le décodage terminé, vous devez également modifier la logique de contrôle du signal de commande. Combiné avec le schéma fonctionnel du système, le tableau logique de contrôle est dessiné comme suit :



INSTRUCTION	nPCSel	RegWr	ALUSrc	ALUCtr	PSREn	MemWr	WrSrc	RegSel	RegAff
ADDi	0	1	1	00	0	0	0	-	0
ADDr	0	1	0	00	0	0	0	0	0
BAL	1	0	0	00	0	0	0	0	0
BLT	N	0	0	00	0	0	0	0	0
CMP	0	0	1	10	1	0	0	0	0
LDR	0	1	1	00	0	0	1	-/0	0
MOV	0	1	1	01	0	0	0	0	0
STR	0	0	1	00	0	1	0	1	1

PARTIE 4 – ASSEMBLAGE ET VALIDATION DU PROCESSEUR

Pour que le code écrit dans le mémoire d'instruction fonctionne, nous initialisons les valeurs de 16-24 à 1-9 lors de l'initialisation du mémoire de données

```
sim:/processor_top_tb/DUT/U_EXEC/U6/mem @ 1459 ps
 0 : 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
 8 : 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
16 : 00000001 00000002 00000003 00000004 00000005 00000006 00000007 00000008
24 : 00000009 00000000 00000000 00000000 00000000 00000000 00000000 00000000
32 : 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
40 : 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
48 : 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
56 : 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
```

Ensuite, le code de Mémoire d'instruction peut être interprété comme suit : R1 pointe d'abord vers mem[16], puis lit à son tour la valeur de mem[16]-mem[24] et s'accumule jusqu'à R2, et lorsque R1 = 0x1A à la fin de la boucle, la valeur finale accumulée est stockée dans mem[R1], c'est-à-dire que le mem[26] final doit être 0x2D.

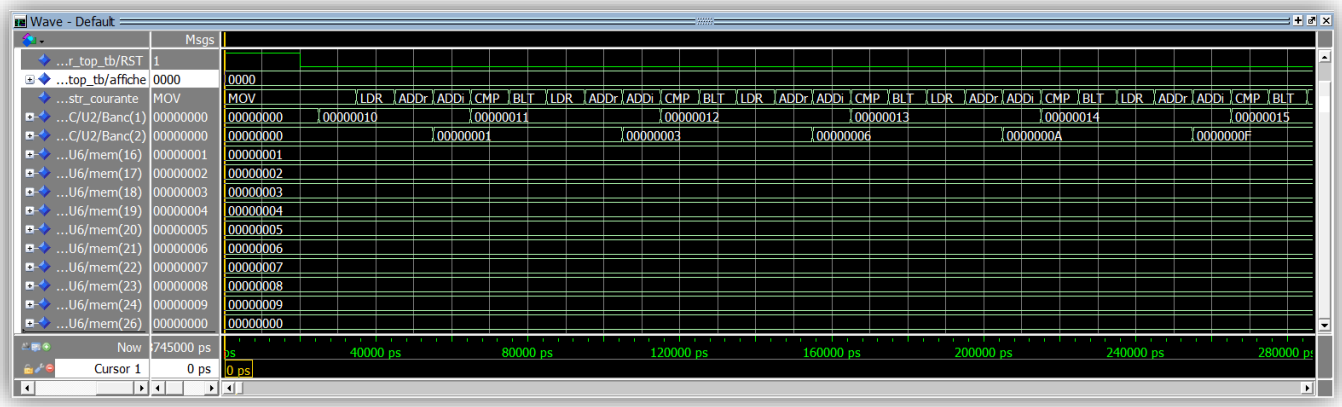


Figure 6 : Simulation de processeur 1

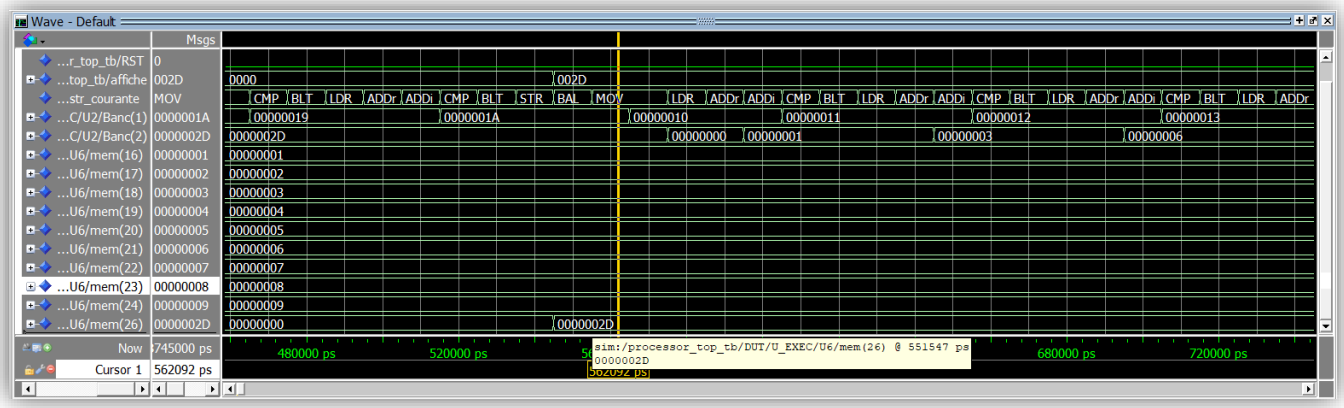
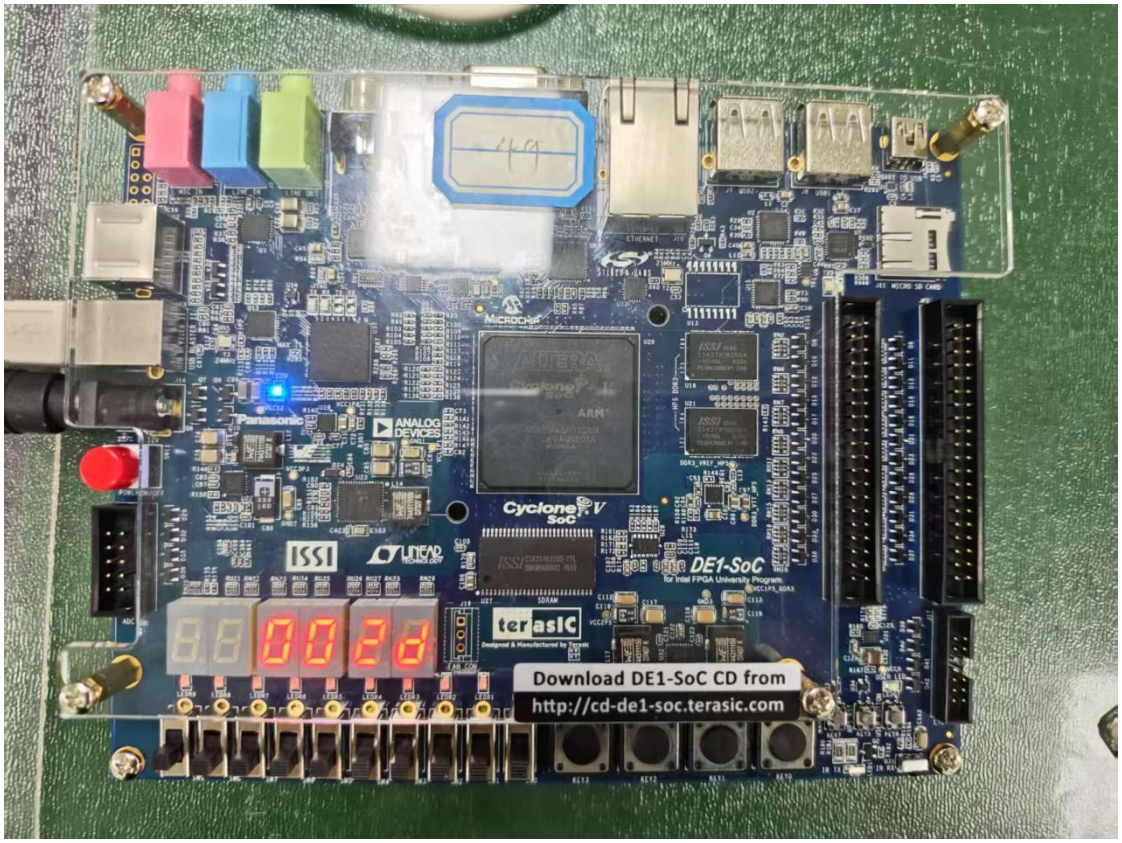


Figure 7 : Simulation de processeur 2

Comme vous pouvez le voir sur le schéma de simulation, la boucle est exécutée correctement, et les valeurs du Mémoire de données sont lues et affectées à R2 après accumulation. On peut voir que le résultat du mem le plus cumulatif[26] est 0x2D

PARTIE 5 : TEST DU PROCESSEUR SUR CARTE FPGA

Après cela, utilisez Processeur comme haut et gravez le code du programme sur la carte. Nous avons extrait une ligne du BUSB et l'avons connectée aux 7 segments de la carte FPGA, et elle a fini par afficher 002D comme prévu.



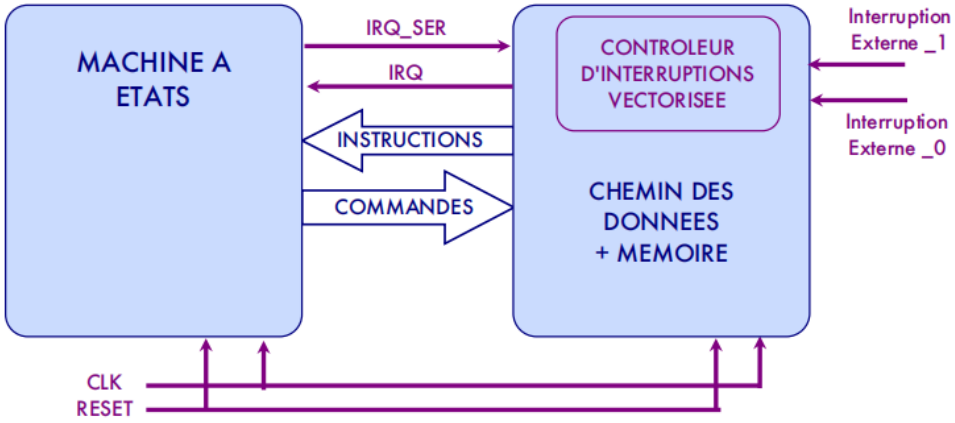
PROCESSEUR MULTI-CYCLE

Processeurs à cycle unique

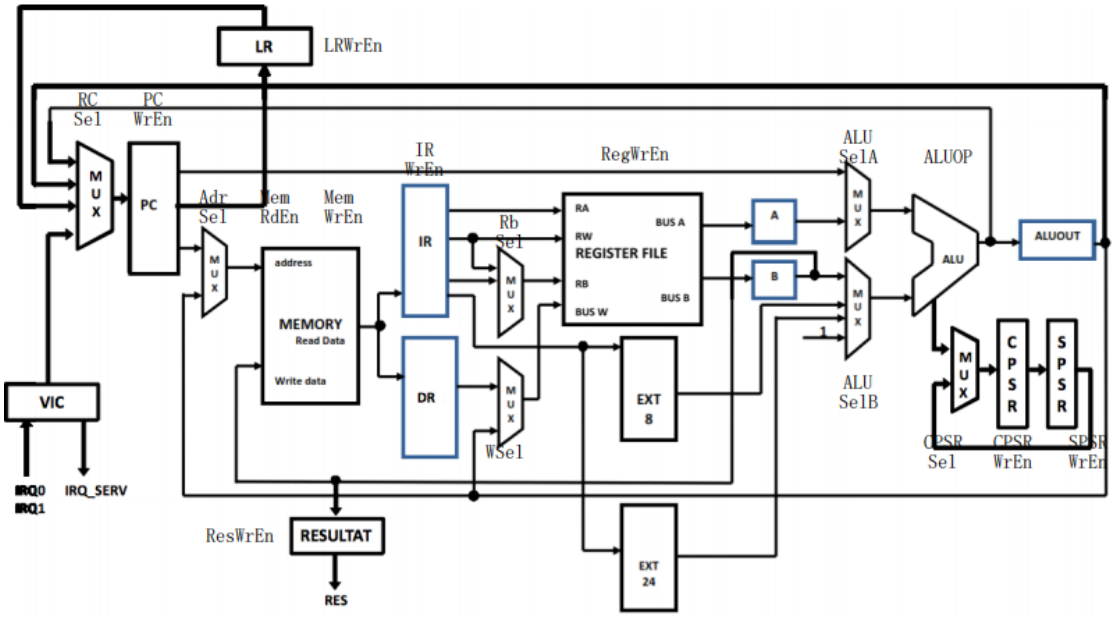
- Chaque instruction est complétée en un cycle d'horloge (IPC = 1).
- La durée du cycle d'horloge est déterminée par l'instruction la plus lente (par exemple, l'instruction d'accès à la mémoire).
- Le chemin d'accès aux données est fixe et toutes les instructions partagent le même chemin matériel.

Processeurs multi-cycles

- Chaque instruction est décomposée en plusieurs petites étapes (telles que le doigté, le décodage, l'exécution, la récupération de mémoire et l'écriture différée), et chaque étape occupe un court cycle d'horloge.
- Le cycle d'horloge est déterminé par le pas le plus lent (généralement plus court que le cycle d'un seul cycle).
- L'exécution étape par étape est réalisée par le biais des instructions de contrôle de la machine d'état (FSM).



PARTIE 1 : FONCTIONNEMENT DU CHEMIN DE DONNEES + MÉMOIRE



Par rapport aux processeurs à cycle unique, les processeurs à cycle multiple ont une structure légèrement plus complexe que les processeurs à cycle unique, et il y a beaucoup plus de registres qui conduisent à des chemins d'exécution différents pour chaque instruction, et le nombre de registres passant par est différent, ce qui conduit finalement à des cycles différents requis pour chaque instruction. Dans le diagramme, nous pouvons voir que le processeur multicycle dispose d'un contrôleur d'interruption vectorisé VIC supplémentaire et d'un registre d'état actuel du processeur (CPSR).

Controleur d'interruption vectorisé (VIC)

Le CIV travaille en :
Le front montant est détecté et l'indicateur de rupture est défini

```
if IRQ0 = '1' and IRQ0_prev = '0' then
    IRQ0_memo <= '1';
end if;
if IRQ1 = '1' and IRQ1_prev = '0' then
    IRQ1_memo <= '1';
end if;
```

Les demandes d'interruption sont faites pour différents types d'interruptions

```
IRQ <= IRQ0_memo or IRQ1_memo;
if IRQ0_memo = '1' then
    VICPC <= x"00000009";
elsif IRQ1_memo = '1' then
    VICPC <= x"00000015";
else4
    VICPC <= x"00000000";
end if;
```

La machine d'état efface l'indicateur d'interruption après avoir accusé réception de l'exécution de l'interruption

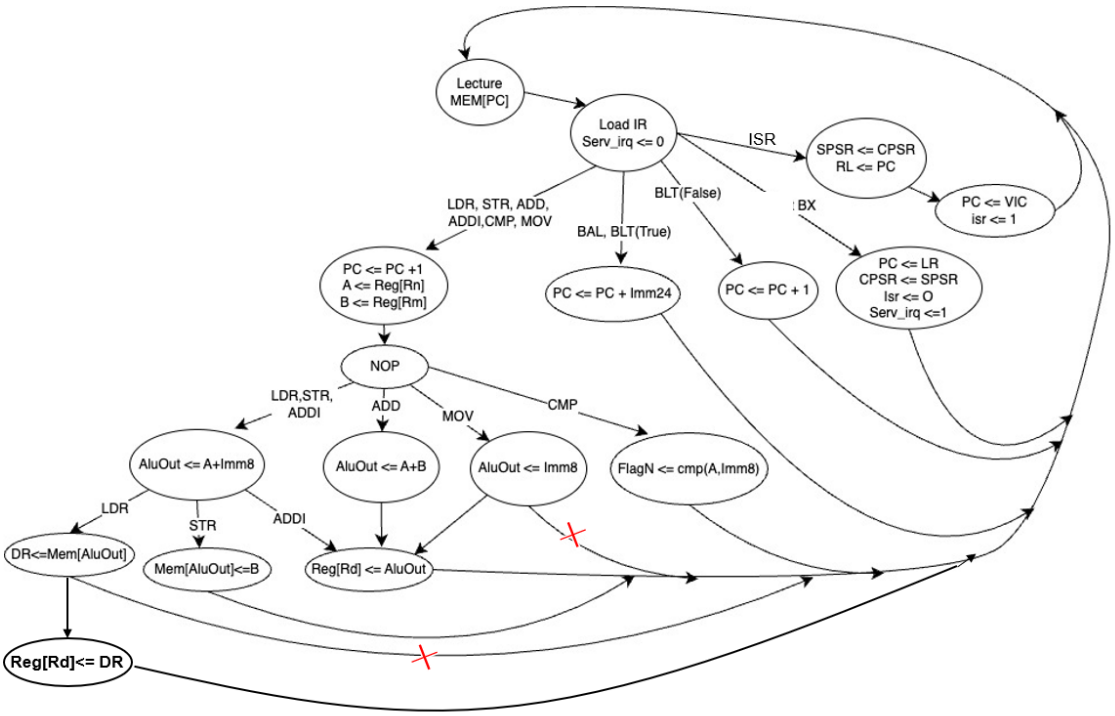
```
if IRQ_SERV = '1' then
    if IRQ0_memo = '1' then
        IRQ0_memo <= '0';
    elsif IRQ1_memo = '1' then
        IRQ1_memo <= '0';
    end if;
end if;
```

Current Processor Status Register (CPSR)

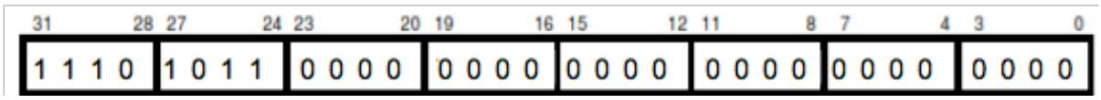
Ce registre 32 bits avec commande de chargement stocke l'état courant du processeur. Mais nous n'utilisons que le chiffre 0 pour indiquer le résultat positif ou négatif de l'opération.

PARTIE 2 : LA MACHINE A ETATS

Notre machine à états a défini au total 16 états afin de satisfaire les exigences de cycle nécessaires à l'exécution de toutes les instructions。



Il n'y a pas de changements majeurs dans la logique de l'analyse des instructions, à l'exception de l'ajout d'une instruction BX qui restaurera l'état de saut précédent



En ce qui concerne le décodage des instructions, étant donné que le contenu lu directement depuis la mémoire peut être du code machine d'instruction ou des données. Bien que les instructions temporisées dans le registre d'instructions (IR) soient toutes des instructions, lors du premier examen de l'instruction par la machine d'état, l'instruction dans le registre d'instructions (IR) n'a pas encore été mise à jour. Cela peut entraîner l'exécution ultérieure de l'ancienne instruction. Par conséquent, lors de la première détermination de l'état de saut d'instruction, on utilise directement l'instruction nouvelle lue depuis la mémoire (inst_new), tandis que pour les opérations ultérieures d'examen des instructions, on utilise l'instruction ancienne du registre d'instructions (IR) (inst_old)..

```
type enum_instruction is (VID,MOV, ADDi, ADDr, CMP, LDR, STR, BAL, BLT_True, BLT_False, BX, ISR);
signal inst_new, inst_old : enum_instruction;
```

```
inst_new <= BX          when inst_mem = "11101011000000000000000000000000" else
            ISR         when irq='1'else
            BLT_True    when inst_mem(27 downto 26) = "10" and inst_mem(31 downto 28) = "1011" and N = '1' else
            BLT_False   when inst_mem(27 downto 26) = "10" and inst_mem(31 downto 28) = "1011" and N = '0' else
            BAL         when inst_mem(27 downto 26) = "10" else
            MOV         when inst_mem(24 downto 21) = "1101" else
            ADDi        when inst_mem(24 downto 21) = "0100" and inst_mem(25) = '1' else
            ADDr        when inst_mem(24 downto 21) = "0100" else
            CMP         when inst_mem(24 downto 21) = "1010" else
            LDR         when inst_mem(24 downto 21) = "0000" and inst_mem(20) = '1' else
            STR         when inst_mem(24 downto 21) = "0000" else

            VID;

inst_old <= BX          when inst_reg = "11101011000000000000000000000000" else
            --ISR         when irq='1'else
            BLT_True    when inst_reg(27 downto 26) = "10" and inst_reg(31 downto 28) = "1011" and N = '1' else
            BLT_False   when inst_reg(27 downto 26) = "10" and inst_reg(31 downto 28) = "1011" and N = '0' else
            BAL         when inst_reg(27 downto 26) = "10" else
            MOV         when inst_reg(24 downto 21) = "1101" else
            ADDi        when inst_reg(24 downto 21) = "0100" and inst_reg(25) = '1' else
            ADDr        when inst_reg(24 downto 21) = "0100" else
            CMP         when inst_reg(24 downto 21) = "1010" else
            LDR         when inst_reg(24 downto 21) = "0000" and inst_reg(20) = '1' else
            STR         when inst_reg(24 downto 21) = "0000" else

            VID;
```

Dans le diagramme de flux de la machine d'état, les opérations correspondant à chaque état indiquent les opérations à effectuer dans cet état. Cependant, en raison du temps d'attente d'un bord montant entre l'assignation du signal d'instruction d'opération et l'exécution de l'opération, pour réaliser une opération dans un état donné, il est nécessaire d'assigner la valeur au signal de commande avant la transition depuis l'état précédent.

```
when E4 =>  -- A+Imm8
    if inst_old = LDR then
        state <= E8;

    elsif inst_old = STR then
        state <= E9;
        AdrSel <= '1';
        MemWrEn <= '1';
        ResWrEn <= '1';

    elsif inst_old = ADDi then
        state <= E10;

    end if;
```

PARTIE 3 : CODAGE VHDL DE LA MACHINE A ETATS

Les sous-programmes d'interruption incrémentent la valeur de l'une des cases mémoire, modifiant ainsi le total en sortie.

Programme principal:

```
0x0   _main :    MOV R1,#0x20    ; --R1 <= 0x20
0x1              MOV R2,#0           ; --R2 <= 0
0x2   _loop :    LDR R0,0(R1)        ; --R0 <= MEM[R1]
0x3              ADD R2,R2,R0    ; --R2 <= R2 + R0
0x4              ADD R1,R1,#1    ; --R1 <= R1 + 1
0x5              CMP R1,0x2A     ; --? R1 = 0x2A
0x6              BLT loop        ; --branchement à _loop si R1
                                 inferieur a 0x2A
       _end   :
0x7              STR R2,0(R1)        ; --MEM[R1] <= R2
0x8              BAL main        ; --branchement à _main

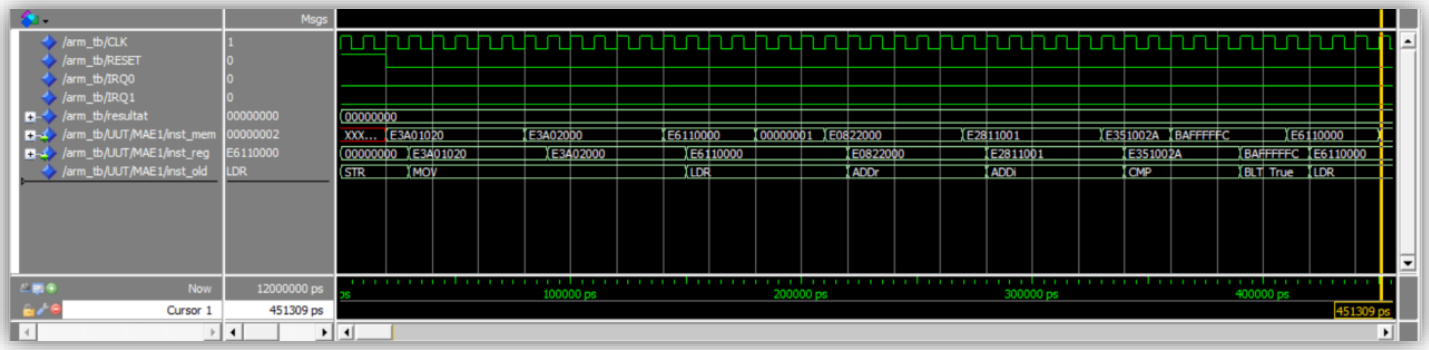
ISR interruption 0 :
  --sauvegarde du contexte - R15 correspond au pointeur de pile
0x9              STR R1,0(R15)       ; --MEM[R15] <= R1
0xA              ADD R15,R15,1       ; --R15 <= R15 + 1
0xB              STR R3,0(R15)       ; --MEM[R15] <= R3
  --traitement
0xC              MOV R3,0x20         ; --R3 <= 0x20
0xD              LDR R1,0(R3)        ; --R1 <= MEM[R3]
0xE              ADD R1,R1,1         ; --R1 <= R1 + 1
0xF              STR R1,0(R3)        ; --MEM[R3] <= R1
  --chargement du context
0x10             LDR R3,0(R15)       ; --R3 <= MEM[R15]
0x11             ADD R15,R15,-1      ; --R15 <= R15 - 1
0x12             LDR R1,0(R15)       ; --R1 <= MEM[R15]
0x13             BX                  ; -- instruction de fin
d'interruption

ISR interruption 1 :
  --sauvegarde du contexte - R15 correspond au pointeur de pile
0x15             STR R4,0(R15)       ; --MEM[R15] <= R4
0x16             ADD R15,R15,1       ; --R15 <= R15 + 1
0x17             STR R5,0(R15)    ; --MEM[R15] <= R5
  --traitement
0x18             MOV R5,0x20         ; --R5 <= 0x20
0x19             LDR R4,0(R5)        ; --R4 <= MEM[R5]
0x1A             ADD R4,R4,2         ; --R4 <= R1 + 2
0x1B             STR R4,0(R5)        ; --MEM[R5] <= R4
  --chargement du contexte
0x1C             LDR R5,0(R15)       ; --R5 <= MEM[R15]
0x1D             ADD R15,R15,-1      ; --R15 <= R15 - 1
0x1E             LDR R4,0(R15)    ; --R4 <= MEM[R15]
0x1F             BX                  ; -- instruction de fin
d'interruption
```

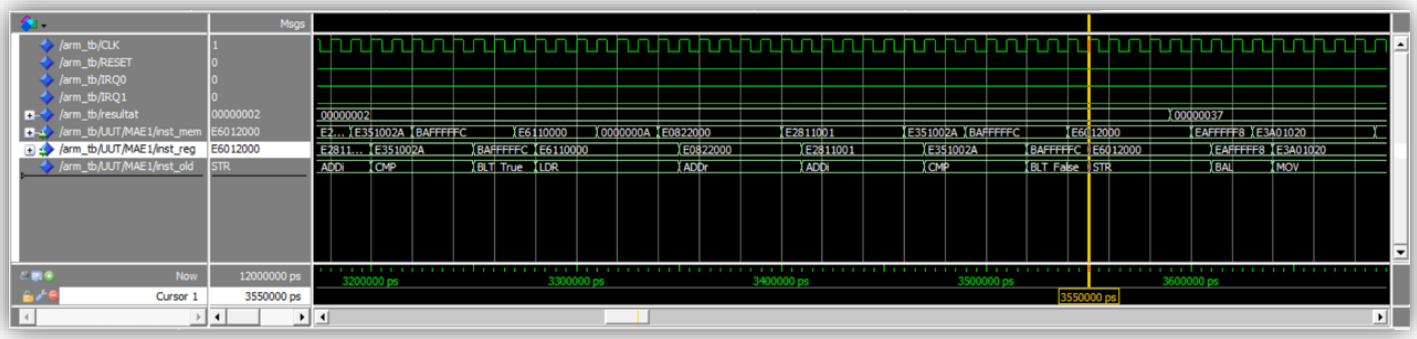
- Pour l'interruption 0 du Service de Routines d'Interruption (ISR) : incrémente le total de 1
- Pour l'interruption 1 du Service de Routines d'Interruption (ISR) : incrémente le total de 2

Après exécution, cette simulation boucle indéfiniment. Voici le déroulement d'une seule itération :

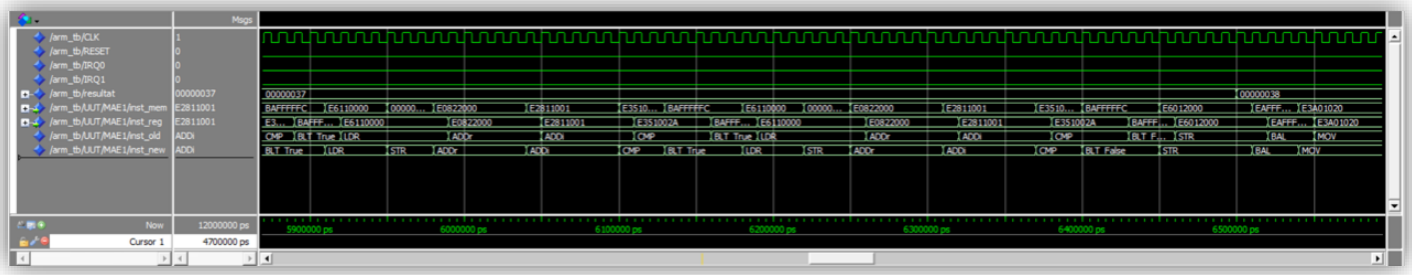
L'instruction de branchement conditionnel (BLT) évalue à Vrai (True), BAL saute à nouveau en boucle.



Après avoir exécuté une série complète de cycles, le résultat de l'accumulation est enregistré et généré dans le résultat. Le résultat de chaque grand cycle est le même que celui de 0x37. BLT FAUSE, BAL REVIENT AU DÉBUT, ET LA SORTIE EST 0x37.



Lorsque l'interruption 0 se produit, le compteur affiche un résultat de 1. La sortie est



0x38.

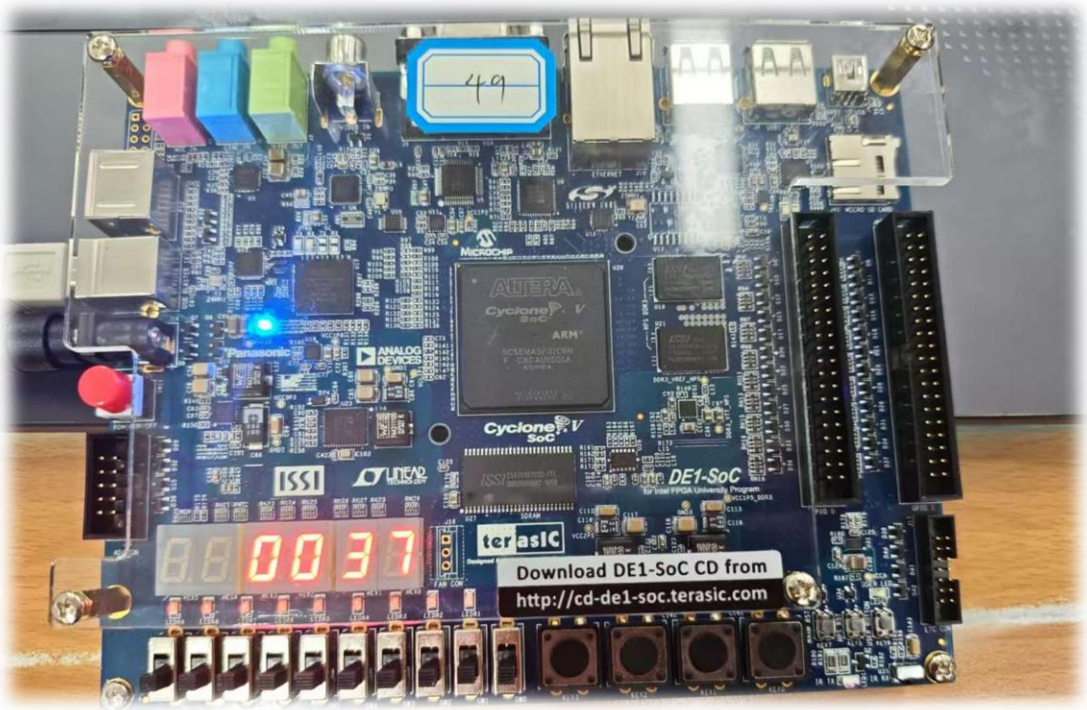
Lorsque l'interruption 1 se produit, le compteur affiche un résultat de 2. La sortie est 0x3a.



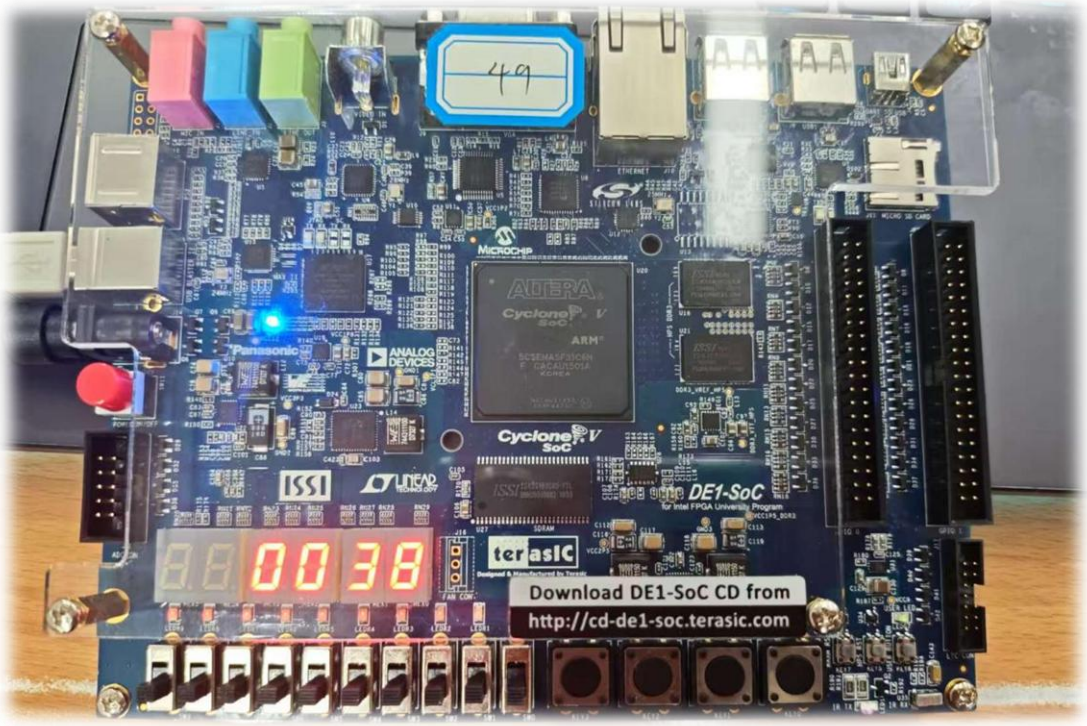
TEST DU PROCESSEUR SUR CARTE FPGA

Après avoir compilé le programme et l'avoir gravé sur le tableau, le résultat est le même que prévu

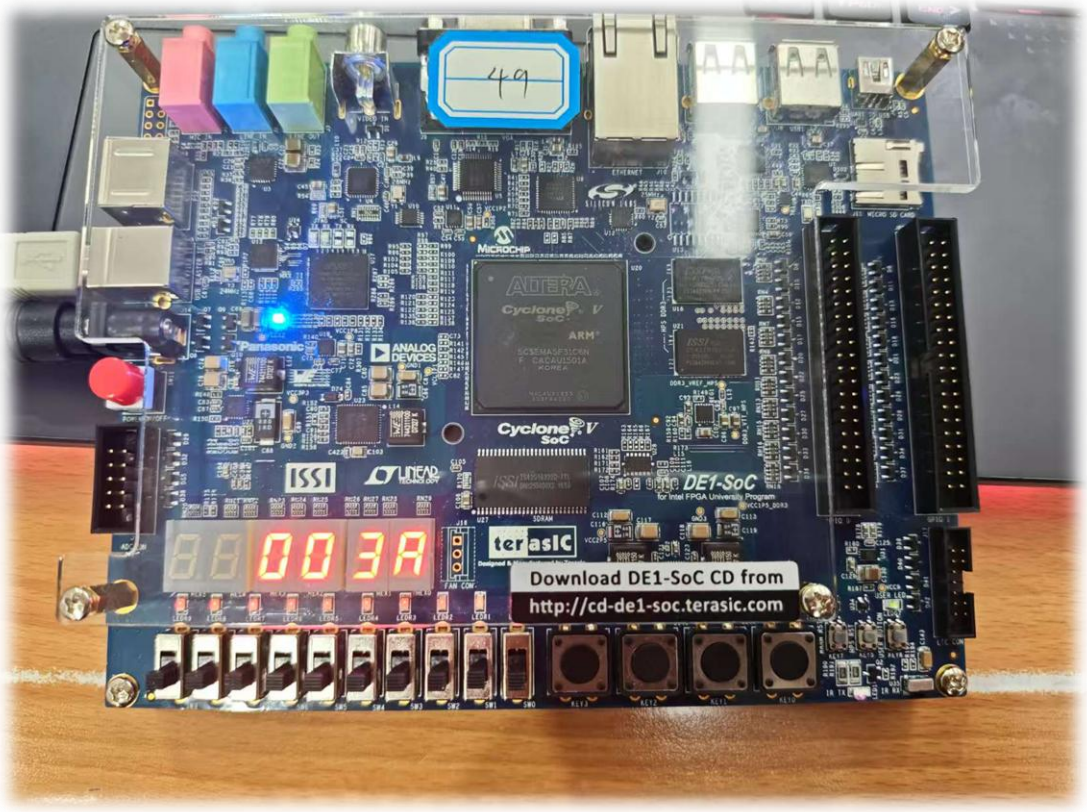
Après avoir exécuté une série complète de cycles, la sortie est 0x37



Lorsque l'interruption 0 se produit, le compteur affiche un résultat de 1. La sortie est 0x38.

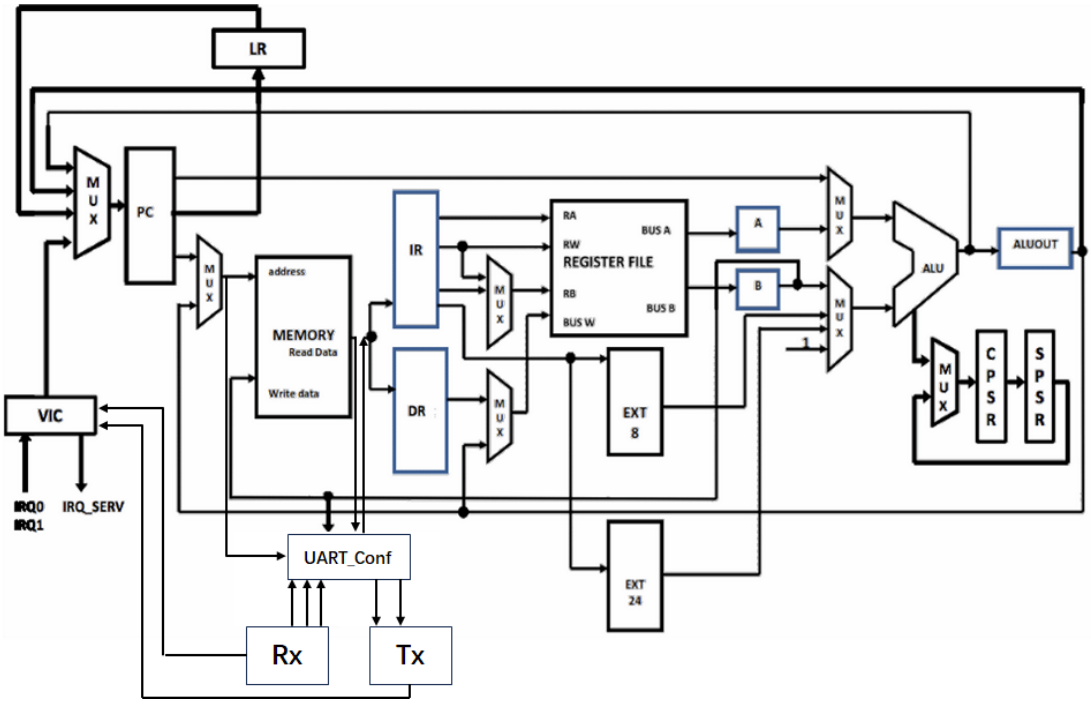


Lorsque l'interruption 1 se produit, le compteur affiche un résultat de 2. La sortie est 0x3a.



PARTIE 4 : RAJOUTER UN PERIPHERIQUE UART AU PROCESSEUR

Sur la base du circuit d'origine, il est légèrement modifié, et le module Uart_Conf, les modules RX et TX sont ajoutés modules RX et TX sont ajoutés 0x38.



Uart_Conf : Lorsque l'adresse d'entrée est 0x40, lisez le contenu écrit par l'instruction STR et transmettez-le au module Tx. Lorsque l'adresse d'entrée est 0x41, remplacez le contenu lu en mémoire par le LDR par le contenu reçu par Rx.

Implémenter l'instruction d'interruption du processus Tx：

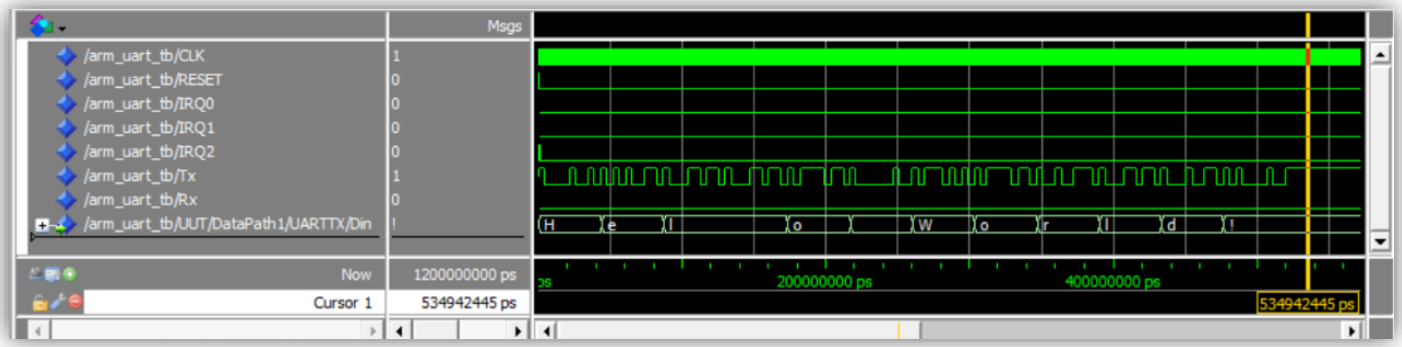
```
                                -- ISR3 : interruption Tx
-- 0x60                          --sauvegarde du contexte - R15 correspond au pointeur de pile
ram_block(96) <=  x"E60F4000"; --          STR R4,0(R15)       ; --MEM[R15] <= R4
ram_block(97) <=  x"E28FF001"; --          ADD R15,R15,1       ; --R15 <= R15 + 1
ram_block(98) <=  x"E60F5000"; --          STR R5,0(R15)   ; --MEM[R15] <= R5
                                        --traitement
ram_block(99) <=  x"E3A04040"; --           MOV R4,0x40        ; --R4 <= 0x40
ram_block(100) <=  x"E3A0507F"; --         MOV R5,0x7F         ; --R5 <= 0x7F
ram_block(101) <=  x"E6156000"; --          LDR R6,0(R5)       ; --R6 <= MEM[R5], R6 = 0x72
ram_block(102) <=  x"E6167000"; --          LDR R7,0(R6)       ; --R7 <= MEM[R6], R7 = 'H'
ram_block(103) <=  x"E2866001"; --          ADD R6,R6,1        ; --R6 <= R6 + 1
ram_block(104) <=  x"E356007F"; --          CMP R6,0x7F        ; -- R6 = 0x7F ?
ram_block(105) <=  x"BA000003"; --          BLT +3    ; --branchement à si R6 inferieur a 0x79
ram_block(106)  <=  x"E3A06072"; --          MOV R6,0x72       ; --R6 <= 0x72  reinit du pointeur
ram_block(107)  <=  x"EA000002"; --          BAL +2        ; --branchement
ram_block(108) <=  x"E6047000"; --          STR R7,0(R4)       ; --MEM[R4] <= R7 ecriture carac dans UART
ram_block(109) <=  x"E6056000"; --          STR R6,0(R5)       ; --MEM[R5] <= R6 mise à jour du pointeur
                                    -- restauration du contexte
ram_block(110) <=  x"E61F5000";--          LDR R5,0(R15)       ; --R5 <= MEM[R15]
ram_block(111) <=  x"E28FF0FF";  --          ADD R15,R15,-1    ; --R15 <= R15 - 1
ram_block(112) <=  x"E61F4000"; --          LDR R4,0(R15)   ; --R4 <= MEM[R15]
ram_block(113) <=  x"EB000000";--          BX                  ; -- instruction de fin d'interruption
```

Implémentez l'instruction d'interruption pour le processus Rx :

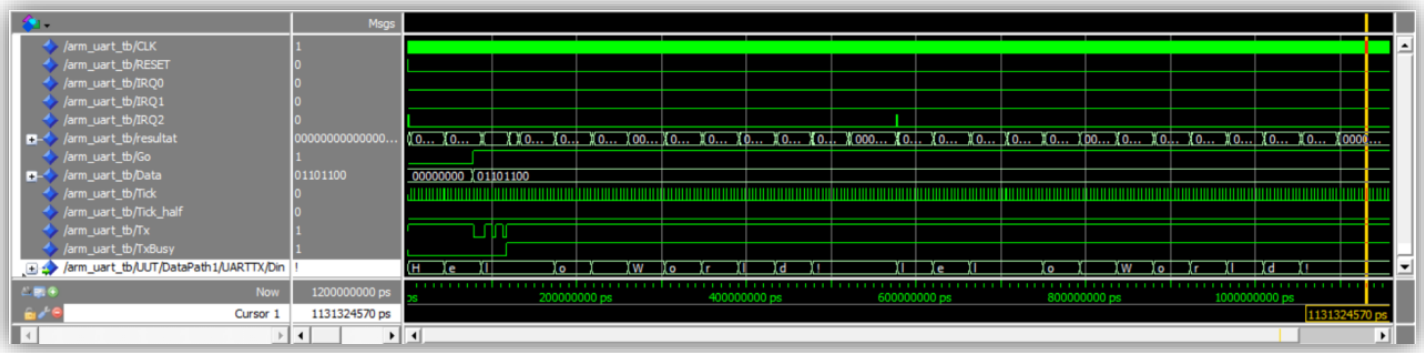
```
                                -- ISR2 : interruption Rx
-- 0x42                                --sauvegarde du contexte - R15 correspond au pointeur de pile
ram_block(66) <=  x"E60F4000"; --           STR R4,0(R15)       ; --MEM[R15] <= R4
ram_block(67) <=  x"E28FF001"; --           ADD R15,R15,1       ; --R15 <= R15 + 1
ram_block(68) <=  x"E60F5000"; --           STR R5,0(R15)   ; --MEM[R15] <= R5
                                          --traitement
ram_block(69) <=  x"E3A04041"; --            MOV R4,0x41        ; --R4 <= 0x41
ram_block(70) <=  x"E3A05053"; --           MOV R5,0x53         ; --R5 <= 0x53
ram_block(71) <=  x"E6156000"; --           LDR R6,0(R5)        ; --R6 <= MEM[R5], R6 = 0x72
ram_block(72) <=  x"E6147000"; --           LDR R7,0(R4)        ; --R7 <= MEM[R4] lecture dans UART
ram_block(73) <=  x"E6067000"; --           STR R7,0(R6)        ; --MEM[R6] <= R7
ram_block(74) <=  x"E2866001"; --           ADD R6,R6,1         ; --R6 <= R6 + 1
ram_block(75) <=  x"E356007E"; --           CMP R6,0x7E         ; -- R6 = 0x7E
ram_block(76) <=  x"BA000002"; --           BLT +2    ; --branchement à si R6 inferieur a 0x79
ram_block(77)  <=  x"E3A06072"; --           MOV R6,0x72        ; --R6 <= 0x72  reinit du pointeur
ram_block(78) <=  x"E6056000"; --           STR R6,0(R5)        ; --MEM[R5] <= R6 mise à jour du pointeur
                                        -- restauration du contexte
ram_block(79) <=  x"E61F5000";--           LDR R5,0(R15)    ; --R5 <= MEM[R15]
ram_block(80) <=  x"E28FF0FF";  --           ADD R15,R15,-1     ; --R15 <= R15 - 1
ram_block(81) <=  x"E61F4000"; --           LDR R4,0(R15)   ; --R4 <= MEM[R15]
ram_block(82) <=  x"EB000000";--           BX              ; -- instruction de fin d'interruption
ram_block(83) <=  x"00000072"; -- 0x53
```

Changements apportés au CIV :
Processus Tx : ajoutez une interruption de saisie de clé, l'adresse de saut est 0x00000060 et l'interruption de clé peut être automatiquement poursuivie en fonction de l'état d'envoi de Tx après l'interruption de la clé, et l'adresse de saut est également 0x00000060 jusqu'à ce que l'envoi soit terminé.
Processus Rx : il peut être automatiquement interrompu en fonction de l'état de réception de Rx, et l'adresse de redirection est 0x00000042.

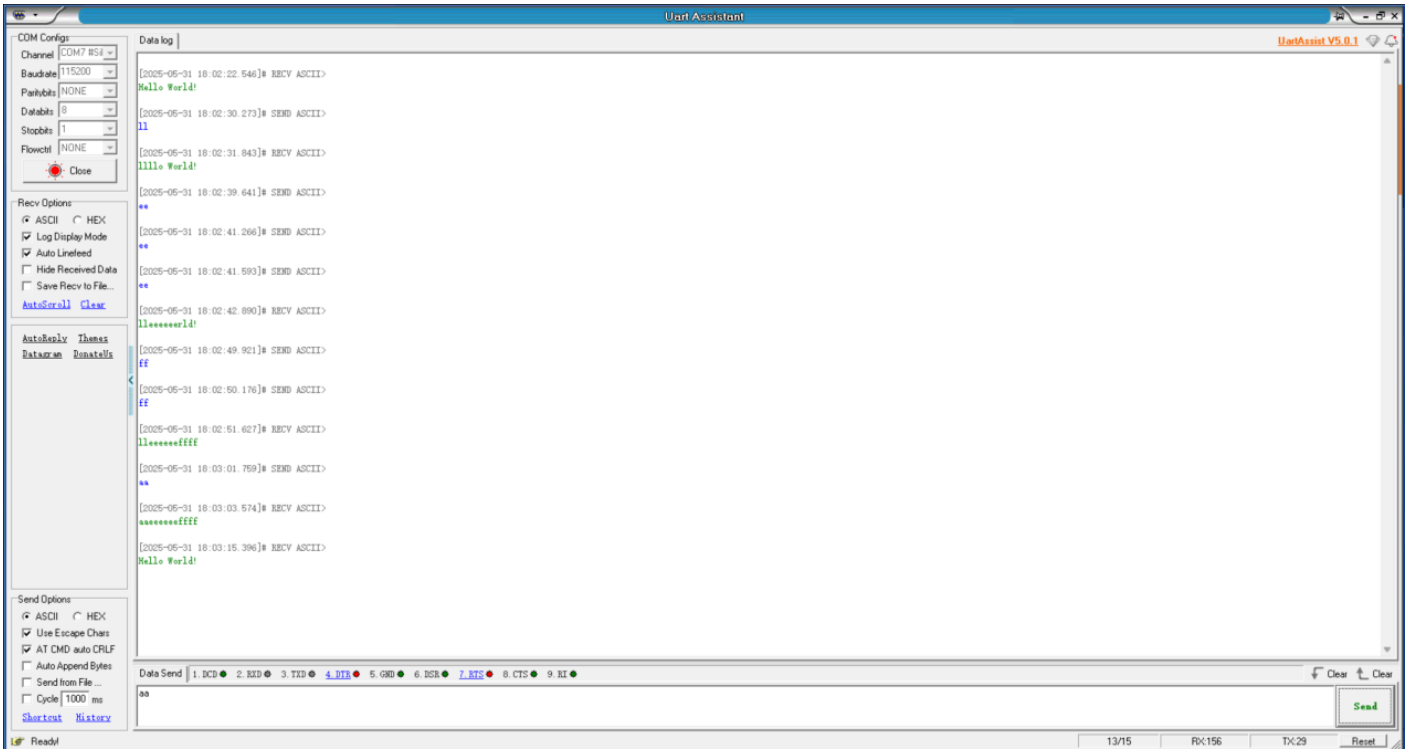
Résultats de la simulation :
Tx : Comme vous pouvez le voir sur l'image, après les entrées IRQ2, « Hello World ! »Correctement transmis à partir de l'interface Din Tx, le signal Tx comme signal de sortie fonctionne également normalement。



Rx : ajoutez un module Tx supplémentaire au banc d'essai pour les tests, transmettez les données entrantes au module Tx et connectez le module Rx au processeur. Après la première interruption de l'IRQ2, les données « 01101100 » sont transmises par Tx, et vous pouvez voir que la chaîne de sortie se transforme en « lello world ! »



Résultats pratiques：



Logiciel utilisé : UartAssistant V5.0.1
Lien de téléchargement : http://www.cmsoft.cn/resource/download/101

Lorsque le bouton est pressé pour la première fois, « Hello World ! » est automatiquement envoyé. "Les chaînes, après que l'ordinateur ait envoyé des caractères, peuvent être reçues et enregistrées dans la mémoire de chaînes, et le contenu démarrera automatiquement à partir de zéro lorsque le contenu sera plein. Après avoir appuyé sur le bouton de réinitialisation, le contenu de la chaîne est réinitialisé sur « Hello World ! »" 。